DIGIZAFE — COMMON MASTER NOTES

Everyone in the team should know this

---

1. THE PROJECT IN ONE SENTENCE

> **DigiZafe is a Zero-Trust Digital Exposure Management Platform that discovers digital exposure, correlates threat intelligence, evaluates risk explainably, and provides privacy-preserving remediation.**

The core idea is:

DISCOVER
↓
UNDERSTAND
↓
SCORE
↓
EXPLAIN
↓
PROTECT

The project's central motivation is to move from simply asking **"Was I breached?"** to answering **"How exposed am I, why, and what should I do?"**

---

2. THE PROBLEM

Traditional security tools are fragmented.

Tool 1 — Breach checker

Was my email breached?
YES / NO

Tool 2 — Password manager

Stores passwords

Tool 3 — OSINT tool

Finds information

But they don't naturally form one complete workflow.

The defense material identifies the main problems as:

- binary breach notifications,
- lack of OSINT correlation,
- lack of explainable scoring,
- fragmented security workflow,
- weak privacy guarantees.

DigiZafe combines them:

Exposure Discovery
+
OSINT Intelligence
+
Risk Assessment
+
Credential Protection
+
Remediation

---

3. RESEARCH GAP

Remember this answer:

> **"Existing systems generally solve individual parts of the problem. Breach services provide exposure lookup, password managers provide secure storage, and OSINT tools provide discovery. DigiZafe attempts to unify exposure detection, contextual risk assessment, privacy-preserving credential protection and remediation in one workflow."**

This is directly aligned with the project's stated research gap.

---

4. WHAT DOES DEMP MEAN?

DEMP = Digital Exposure Management Platform

Think of it as:

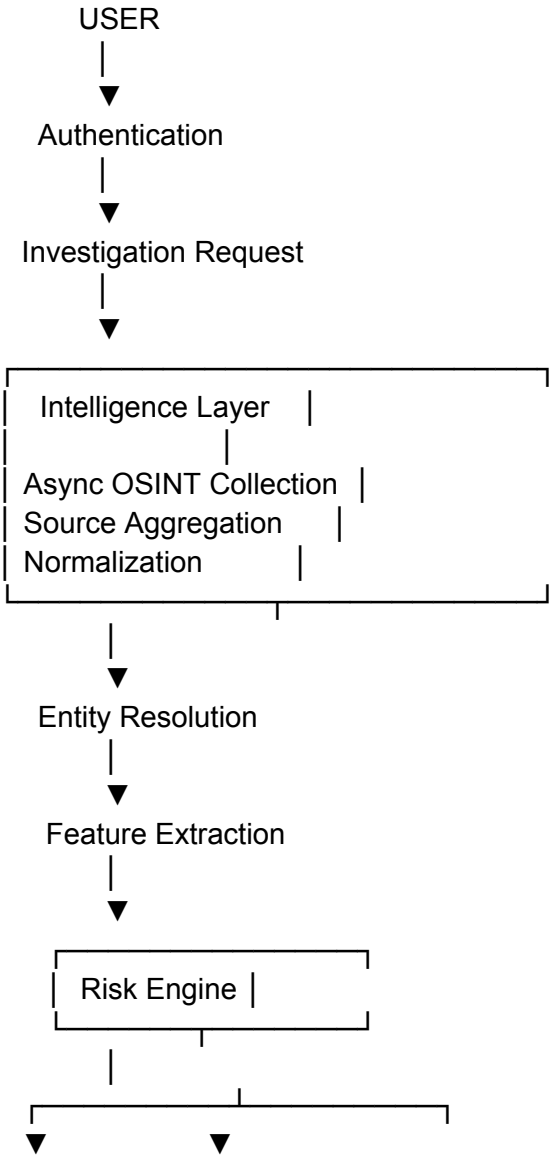
Digital Exposure
↓
Management
↓
Detection + Assessment + Mitigation

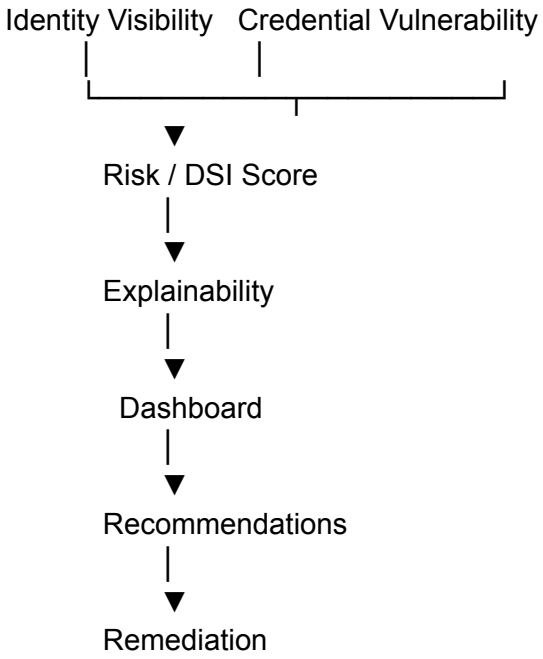
It is broader than a simple breach checker.

---

5. COMPLETE ARCHITECTURE

Everyone should be able to draw this.





The defense deck describes the architecture as:

Presentation Layer → Streamlit
Application Layer → FastAPI
Intelligence Layer → Async OSINT
Risk Engine → Deterministic DSI
Persistence → PostgreSQL + Redis

---

6. THE FIVE MAJOR LAYERS

Layer 1 — Presentation

Streamlit

Responsible for:

- user interaction,
- investigation input,
- displaying results,
- risk visualization,
- recommendations.

Layer 2 — Application

FastAPI

Responsible for:

- API endpoints,
- orchestration,
- business logic,
- authentication/RBAC,
- communication between components.

Layer 3 — Intelligence

Responsible for:

- OSINT collection,
- external intelligence,
- source aggregation,
- normalization.

Layer 4 — Risk Engine

Responsible for:

Features
↓
Risk calculation
↓
DSI / Risk
↓
Explanation

Layer 5 — Persistence

PostgreSQL

Stores structured application data.

Redis

Used for caching and faster repeated access.

---

7. END-TO-END WORKFLOW

This is probably the **single most important common topic**.

Memorize:

1. User logs in
   ↓
2. User submits identifier
   ↓
3. Investigation begins
   ↓
4. Threat intelligence is collected
   ↓
5. Results are normalized
   ↓
6. Entities/evidence are correlated
   ↓
7. Features are extracted
   ↓
8. Risk is calculated
   ↓
9. Explanation is generated
   ↓
10. Dashboard displays result
   ↓
11. User receives mitigation advice

The defense deck summarizes the operational workflow as authentication → investigation → concurrent threat collection → normalization/DSI calculation → dashboard/remediation.

---

8. WHAT IS OSINT?

OSINT = Open-Source Intelligence

Information obtained from publicly accessible sources.

Examples:

Public profiles
Usernames
Domains
DNS information
Public websites
Publicly indexed information
Breach intelligence

Important:

> **OSINT provides raw intelligence; it does not automatically mean that the information belongs to the target.**

Therefore correlation and evidence handling are important.

---

9. WHY ASYNCHRONOUS OSINT?

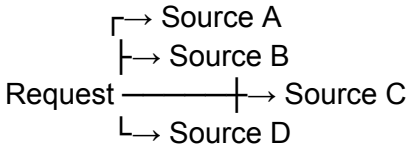
Suppose we query four sources.

Sequential

Source A
↓
wait
↓
Source B
↓
wait
↓
Source C
↓
wait
↓
Source D

Slow.

Asynchronous



Multiple network requests can progress concurrently.

Viva answer:

> **"OSINT collection is network-bound, so asynchronous execution allows independent sources to be queried concurrently, reducing unnecessary waiting time and improving responsiveness."**

---

10. WHY NORMALIZATION?

Different sources return different formats.

For example:

Source A → breach_count
Source B → breaches
Source C → exposureCount

We convert them into a common internal representation:

breach_count

Why?

Because the risk engine should not need to understand every external API format.

---

11. ENTITY RESOLUTION

Different sources may refer to the same entity.

Example:

Source A → username: yuva123
Source B → handle: yuva123
Source C → profile URL: /yuva123

The system needs to determine whether these observations can be correlated.

Simple answer:

Entity resolution is the process of correlating observations from different sources to determine whether they represent the same candidate entity.

---

12. WHAT IS RISK SCORING?

Raw data isn't immediately useful.

Example:

Breach count = 7
Recent breach = Yes
Password exposed = Yes
Severity = High

The user needs:

HIGH RISK

plus:

WHY?
WHAT SHOULD I DO?

That's the purpose of risk scoring.

---

13. TWO RISK DIMENSIONS

The deterministic architecture separates risk into:

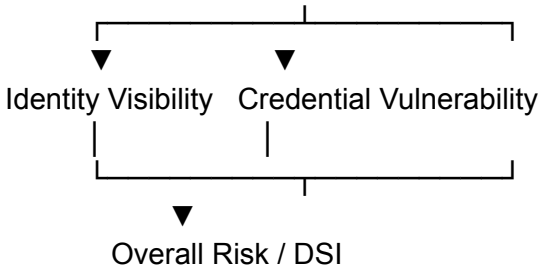
1. Identity Visibility

How exposed the person's digital identity is.

2. Credential Vulnerability

How vulnerable their credentials are.

DIGITAL EXPOSURE



The research explicitly describes this dual-axis methodology.

---

14. DIGITAL SAFETY INDEX

In the deterministic version:

DSI = 100 − (0.40 × Videntity + 0.60 × Vcred)

Meaning:

40% → Identity Visibility
60% → Credential Vulnerability

Important:

DSI is a risk/safety index, not automatically a probability of being hacked.

---

15. WHAT AFFECTS CREDENTIAL VULNERABILITY?

The project materials identify factors such as:

Breach severity
Breach frequency
Recency
Password exposure
Repeated compromise

Another ML-oriented version explicitly uses six features:

Breach count
Password exposure
Severity
Source type
Recency
Frequency

IMPORTANT VERSION NOTE

There are multiple DigiZafe versions in your files.

Therefore:

> **Don't mix the six-feature ML model with the dual-axis deterministic DSI model unless the professor asks about different versions of the project.**

---

16. WHY RECENCY?

A breach from years ago and a breach from last week shouldn't necessarily carry identical urgency.

Conceptually:

Recent exposure
↓
Higher urgency

Old exposure
↓
Reduced influence

This is implemented through temporal decay in the deterministic versions.

---

17. WHY LOGARITHMIC SCALING?

If a user has:

5 breaches

and another has:

500 breaches

linear scaling can allow the count to dominate the score.

Logarithmic scaling compresses large values.

Viva:

> **"Logarithmic scaling prevents very large exposure counts from disproportionately dominating the risk score and helps avoid alert saturation."**

The V4 architecture explicitly identifies logarithmic risk computation as part of its pipeline.

---

18. WHY EXPLAINABILITY?

Never say:

> "Our AI gives a score."

Instead say:

Score
↓
Contributing factors
↓
Explanation
↓
Recommended action

Example:

Risk = HIGH

Reasons:
• Recent exposure
• Password involved
• Repeated compromise

Action:
Change affected password
Enable MFA

Viva answer:

> **"Explainability is necessary because a score without its underlying reasons is difficult for a user or security analyst to act upon."**

---

19. WHY REDIS?

Everyone should know:

```
Request
↓
Redis?
 ├── YES → Return cached result
 └── NO  → Perform investigation
            ↓
          Store result
```

Benefits:

- lower latency,
- fewer repeated queries,
- reduced external dependency,
- improved responsiveness.

---

20. WHY POSTGRESQL?

PostgreSQL provides persistent structured storage for application and investigation-related data.

Think:

```
Redis      → fast temporary/cache access
PostgreSQL → persistent structured storage
```

---

21. WHY FASTAPI?

FastAPI is used as the application/API layer.

Responsibilities include:

API endpoints
Request handling
Orchestration
Business logic
Authentication

---

22. WHY STREAMLIT?

Streamlit provides a rapid interactive interface for the research prototype.

It allows:

Input
↓
Investigation
↓
Results
↓
Charts/cards
↓
Recommendations

The defense architecture explicitly identifies Streamlit as the presentation layer.

---

23. ZERO TRUST — EVERYONE MUST KNOW

This is one of the **most likely viva questions**.

What does Zero Trust mean here?

The important principle is:

> **The backend should not automatically be trusted with plaintext credentials.**

Therefore:

User enters credential
↓
Browser derives key
↓
Encryption
↓
Ciphertext
↓
Backend

Not:

Password
↓
Backend
↓
Encrypt

---

24. PBKDF2

PBKDF2 = Password-Based Key Derivation Function 2

It derives a cryptographic key from a password.

Conceptually:

Master Password
+
Salt
+
Iterations
↓
Derived Key

The documented Zero-Trust architecture uses PBKDF2 before AES-GCM encryption.

---

25. AES-GCM

AES

Symmetric encryption algorithm.

GCM

Provides authenticated encryption.

Therefore:

Plaintext
↓
AES-GCM
↓
Ciphertext + authentication information

The important property for the project:

> The backend stores/transports encrypted data rather than plaintext vault contents.

---

26. WHY CLIENT-SIDE ENCRYPTION?

Because:

Backend compromise
↓
Attacker obtains database
↓
Should NOT obtain plaintext passwords

if encryption is properly implemented client-side.

The project explicitly identifies preventing backend access to plaintext vault contents as a core Zero-Trust property.

---

27. K-ANONYMITY

Another **must-know** concept.

Suppose the user wants to check whether a password has appeared in a breach.

You don't want to send:

actual password

to an external service.

Instead:

Password
↓
Hash
↓
Send only partial hash information
↓
External service

The Pwned Passwords style approach uses prefix-based querying so the complete password hash isn't disclosed.

Viva:

> **"K-anonymity reduces information disclosure by allowing a password exposure check without sending the complete password or complete hash to the external service."**

---

28. PASSWORD VAULT

The password side of DigiZafe provides:

Password generation
+
Credential storage
+
Vault health analysis

The ML-oriented prototype describes high-entropy password generation, encrypted credential storage and identification of weak/reused credentials.

---

29. WHAT IS VAULT HEALTH?

It can identify issues such as:

Weak password
Reused password
Potentially exposed password

The purpose is not just storage.

It is:

STORE
+
ASSESS
+
IMPROVE

---

30. AUTHENTICATION VS AUTHORIZATION

Everyone must know the difference.

Authentication

> **Who are you?**

Login
Password
MFA

Authorization

> **What are you allowed to do?**

Role
↓
Permissions

---

31. RBAC

RBAC = Role-Based Access Control

Concept:

User
↓
Role
↓
Permissions

The project's dual-persona architecture uses RBAC to separate enterprise and personal interfaces in the relevant version.

---

32. WHAT MAKES DIGIZAFE DIFFERENT?

Memorize these five:

1. Unified exposure discovery
2. OSINT aggregation
3. Explainable risk scoring
4. Zero-Trust credential protection
5. Actionable remediation

The presentation lists essentially this combination as the project's contribution.

---

33. TRADITIONAL VS DIGIZAFE

Traditional approach	DigiZafe
Breached / not breached	Contextual risk
Separate OSINT	Integrated intelligence
Separate password manager	Integrated protection

Raw information	Correlated information
Numerical/limited output	Explainable output
Detection	Detection → mitigation
Backend trust	Zero-Trust credential handling

---

34. WHAT HAPPENS IF AN API FAILS?

Don't say:

> "The system crashes."

Better:

> **"Because the architecture is modular, an unavailable intelligence source can be handled independently and the system can continue with the available evidence, while the dependency and reduced coverage should be communicated."**

The defense material explicitly recognizes external-provider availability and third-party rate limits as limitations.

---

35. WHAT HAPPENS IF OSINT DATA IS INCOMPLETE?

Remember:

No evidence ≠ No exposure

It may simply mean:

Source unavailable
Source doesn't index target
Data is stale

Therefore risk should be interpreted based on available evidence.

---

36. WHAT ARE THE LIMITATIONS?

Everyone should know these.

1. External source dependency

The system relies on publicly available intelligence.

2. WAF/CAPTCHA limitations

Automated collection can be restricted.

3. API rate limits

Third-party providers can limit request volume.

4. Deterministic scoring

It does not automatically learn new behavioral patterns.

5. User remediation

The system can recommend actions, but the user still needs to perform them.

These limitations are explicitly documented in the defense deck.

---

37. HOW DO YOU HANDLE CAPTCHA?

This is a trap question.

Don't say:

> "We bypass CAPTCHA."

Say:

> **"We don't attempt to bypass CAPTCHAs. We respect the source's access controls and prefer official APIs or permitted access mechanisms where available."**

The defense material explicitly gives this answer.

---

38. FUTURE WORK

Know at least these:

Adaptive anomaly detection
Continuous exposure monitoring
MFA
PostgreSQL Row-Level Security
Browser extension
Automated privacy/takedown workflows
Improved explainability
Modern decoupled frontend

The defense deck lists ML-based anomaly detection, continuous monitoring, MFA, RLS and a browser extension.

---

39. WHY BROWSER EXTENSION?

Potential answer:

> **"A browser extension could provide real-time phishing protection, contextual credential safety and convenient vault access while the user is browsing."**

---

40. WHAT IS THE BIGGEST ADVANTAGE?

Memorize:

> **"The biggest advantage is integration: DigiZafe connects active exposure discovery with explainable risk assessment and Zero-Trust credential protection in one workflow."**

This is consistent with the defense material's stated advantage.

---

41. WHAT IS THE MOST DIFFICULT PART?

The defense deck identifies:

> **Integrating asynchronous OSINT collection with a deterministic scoring engine in real time.**

This is a good common answer.

---

42. WHAT IS THE MOST IMPORTANT SECURITY PROPERTY?

Confidentiality of credentials.

Especially:

Backend
×
plaintext credentials

The Zero-Trust design ensures credentials are encrypted client-side before transmission.

---

43. WHAT IS THE MOST IMPORTANT ANALYTICAL PROPERTY?

Explainability.

The system shouldn't merely say:

Risk = 82

It should say:

Risk = 82
because:

recent breach
password exposure
repeated compromise

---

44. WHAT IS THE MOST IMPORTANT SYSTEM PROPERTY?

Modularity.

OSINT
↓
Risk Engine
↓
Dashboard

can evolve independently.

For example, you can change the risk model without completely rewriting the OSINT collection layer.

The defense presentation specifically notes the separation of the Risk Engine from the Intelligence Layer to permit independent evolution.

---

45. ONE COMPLETE EXAMPLE

Everyone should be able to explain this example.

Suppose:

User:
example@email.com

Step 1

User authenticates.

Step 2

Starts an investigation.

Step 3

System collects exposure intelligence.

Step 4

Results are normalized.

Step 5

Features are extracted:

Breach count
Severity
Recency
Password exposure
Frequency

Step 6

Risk engine evaluates them.

Step 7

System obtains:

Risk = HIGH

Step 8

Explanation:

Recent exposure
+
Credential involved
+
Repeated compromise

Step 9

Dashboard displays:

HIGH RISK

Step 10

Recommendation:

Change password
Avoid reuse
Enable MFA
Review affected accounts

Final objective:

DISCOVER → UNDERSTAND → ACT

---

46. TEAM ASSIST SYSTEM

This is particularly important for what you asked: **one member should be able to assist another.**

Imagine Professor asks Member 2 something that actually belongs to Member 5.

Member 2 shouldn't say:

> "That is not my part."

Instead:

> **"That component is primarily handled in Member 5's area, but I can explain the overall flow. The intelligence layer passes normalized exposure features to the risk engine, where the exposure is evaluated and then presented through the dashboard."**

Then Member 5 can continue:

> **"Yes, and specifically the risk engine evaluates the relevant risk dimensions and converts them into an interpretable score and recommendations."**

This makes the team appear **integrated rather than divided into five isolated projects.**

---

47. HAND-OFF PHRASES

Everyone should memorize these.

If another member needs to continue:

> **"This connects directly to the next stage, which [Member] worked on. I'll briefly explain the interface between the two components."**

If asked about another module:

> **"At a high level, that module receives the output from our component and performs the next stage of the pipeline."**

If you don't remember an implementation detail:

> **"At the architectural level, its responsibility is ____. The implementation-specific details were handled in that module."**

If another member corrects/adds something:

> **"Yes, exactly. To connect that with my component, the output is then passed to..."**

Never say:

❌ "I don't know."

Instead:

> **"I don't want to give an incorrect implementation detail, but architecturally the component works as..."**

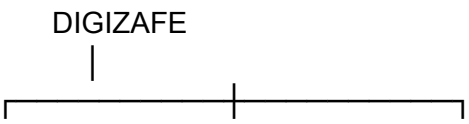
---

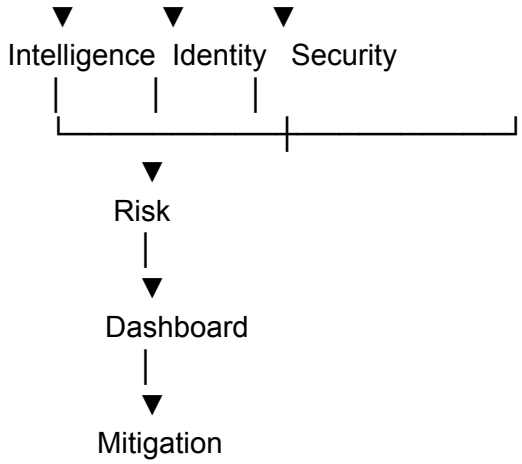
48. HOW THE FIVE MEMBERS SHOULD THINK

Don't think:

Member 1 → my project
Member 2 → my project
Member 3 → my project

Think:





All five people are explaining **one system**.

---

49. 15 QUESTIONS EVERY MEMBER MUST ANSWER

Before the viva, every member should be able to answer these without help:

1. **What is DigiZafe?**
2. **What problem does it solve?**
3. **What is the research gap?**
4. **Why isn't HIBP alone sufficient?**
5. **What is OSINT?**
6. **Why asynchronous collection?**
7. **What is entity resolution?**
8. **What is the risk engine?**
9. **What are the two risk dimensions?**
10. **What is DSI?**
11. **Why Zero Trust?**
12. **Why PBKDF2 + AES-GCM?**
13. **What is k-anonymity?**
14. **Why Redis + PostgreSQL?**
15. **What are the limitations and future work?**

If everyone can answer these, the team can cover most general questions.

---

50. 30-SECOND PROJECT INTRODUCTION

Memorize this collectively:

> **"DigiZafe is a Zero-Trust Digital Exposure Management Platform developed to address the limitations of fragmented cybersecurity tools. It combines exposure discovery, asynchronous OSINT aggregation, contextual and explainable risk assessment, privacy-preserving password verification, and client-side encrypted credential protection. Instead of simply telling the user whether an identifier was breached, DigiZafe evaluates the exposure, explains the associated risk and provides actionable remediation."**

This reflects the project's documented architecture and objectives.

---

51. 10-SECOND VERSION

If the professor suddenly asks:

> "What exactly did you build?"

Say:

> **"We built a platform that discovers digital exposure, evaluates how risky that exposure is, explains why, and provides secure remediation."**

---

52. FINAL-DAY ULTRA-SHORT REVISION SHEET

DIGIZAFE

Problem:

Fragmented security tools
+
Binary breach results
+

No contextual risk
+
No integrated remediation

Solution:

Exposure
→ OSINT
→ Correlation
→ Risk
→ Explanation
→ Protection

Architecture:

Streamlit
↓
FastAPI
↓
Async Intelligence
↓
Normalization
↓
Risk Engine
↓
PostgreSQL + Redis

Risk:

Identity Visibility
+
Credential Vulnerability
→
DSI

Security:

Client-side encryption
PBKDF2
AES-GCM
k-Anonymity
RBAC

Performance:

Async requests
+
Redis caching

Output:

Score
+
Severity
+
Explanation
+
Recommendation

Limitations:

OSINT availability
WAF/CAPTCHA
API rate limits
Deterministic model
User-dependent remediation

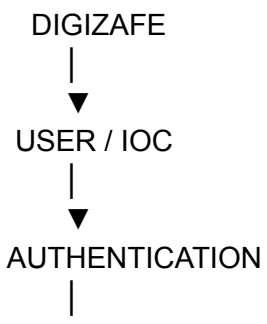
Future:

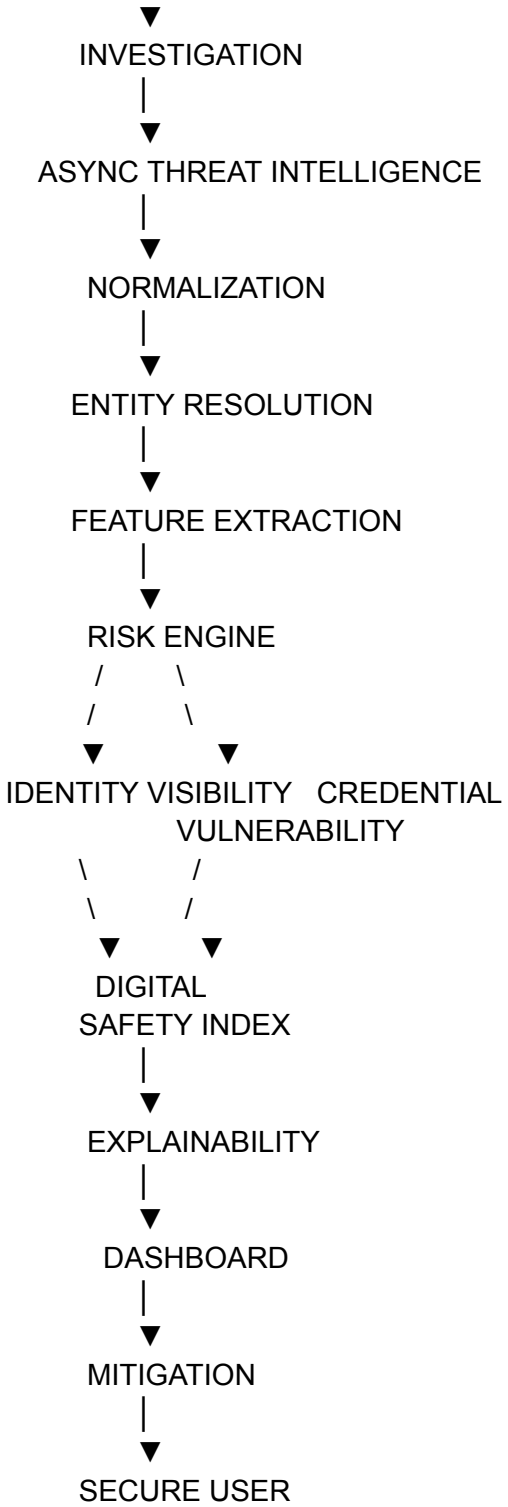
ML/anomaly detection
Continuous monitoring
MFA
RLS
Browser extension

---

53. THE ONE DIAGRAM TO MEMORIZE

If you can reproduce only **one diagram in the viva**, reproduce this:





This single diagram connects **every member's work**.

One important note about your project versions

Your Library currently contains several DigiZafe versions: the deterministic **dual-axis DSI** architecture, the **ML/six-feature** prototype, and a later V4 architecture with logarithmic scoring, zero-knowledge vault concepts, etc.

For the actual viva, **the team should agree on one "official implementation story."** Otherwise one member may say *"we use Gradient Boosting"* while another says *"our scoring is completely deterministic,"* which a professor can immediately notice.

The safest common story from the main defense material is:

> **DigiZafe is a modular Zero-Trust DEMP with asynchronous intelligence collection, normalization, a deterministic dual-axis risk assessment/DSI, explainable dashboard output, and client-side credential protection.**

Then discuss the ML/six-feature version **only when specifically asked about the ML experiment or the particular prototype version**. This keeps the team's answers internally consistent.